

assignment3

Assignment #3: 20260311 cs201 Mock Exam

Updated 2026-03-17 20:20 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

作业的各项评分细则及对应的得分

标准	等级	得分
按时提交	完全按时提交：1分 提交有请假说明：0.5分 未提交：0分	1 分
源码、耗时（可选）、解题思路（可选）	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于2个：0分	1 分
AC代码截图	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于：0分	1 分
清晰头像、PDF文件、MD/DOC附件	包含清晰的Canvas头像、PDF文件以及MD或DOC格式的附件：1分 缺少上述三项中的任意一项：0.5分 缺失两项或以上：0分	1 分
学习总结和个人收获	提交了学习总结和个人收获：1分 未提交学习总结或内容不详：0分	1 分
总得分： 5	总分满分：5分	

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge， Codeforces， LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. 课程平台：

课程网站位于Canvas平台（<https://pku.instructure.com>）。该平台将在第2周选课结束后正式启用。在平台启用前，请先完成作业并将作业妥善保存。待Canvas平台激活后，再上传你的作业。

3. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
4. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E20742:泰波拿契數

implementation, <http://cs101.openjudge.cn/practice/20742/>

思路：

按题意实现

代码：

```
n = int(input())
if n <= 2:
    print(1)
else:
    a, b, c = 0, 1, 1
    for i in range(3, n + 1):
        a, b, c = b, c, a + b + c
    print(c)
```

代码运行截图（至少包含有"Accepted"）

#51973537提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
n = int(input())
if n <= 2:
    print(1)
else:
    a, b, c = 0, 1, 1
    for i in range(3, n + 1):
        a, b, c = b, c, a + b + c
    print(c)
```

基本信息

#: 51973537
题目: 20742
提交人: 25n2500011619
内存: 3632kB
时间: 27ms
语言: Python3
提交时间: 2026-03-14 16:25:03

E30571.十进制整数的反码

bit manipulation, <http://cs101.openjudge.cn/practice/E30571/>

思路：

求反码可以用 $(1 \ll n.\text{bit_length}()) - 1 - n$ ，注意特判 0

代码：

```
n = int(input())
if n == 0:
    print(1)
else:
    print((1 << n.bit_length()) - 1 - n)
```

代码运行截图（至少包含有"Accepted"）

#51973577提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
n = int(input())
if n == 0:
    print(1)
else:
    print((1 << n.bit_length()) - 1 - n)
```

基本信息

#: 51973577

题目: E30571

提交人: 25n2500011619

内存: 3856kB

时间: 24ms

语言: Python3

提交时间: 2026-03-14 16:27:56

E29950:稳定的符文序列

two pointers, <http://cs101.openjudge.cn/practice/E29950>

思路：

使用滑动窗口，区间 $[i, j)$ 中每个字母的个数cnt不能大于1

代码：

```
s = input()
n = len(s)
cnt = [0] * 26
i, j = 0, 0
ans = 0
while j < n:
    while j < n and cnt[ord(s[j]) - 97] == 0:
        cnt[ord(s[j]) - 97] += 1
        j += 1
    ans = max(ans, j - i)
    cnt[ord(s[i]) - 97] -= 1
```

```
i += 1
print(ans)
```

代码运行截图（至少包含有"Accepted"）

#51984224提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
s = input()
n = len(s)
cnt = [0] * 26
i, j = 0, 0
ans = 0
while j < n:
    while j < n and cnt[ord(s[j]) - 97] == 0:
        cnt[ord(s[j]) - 97] += 1
        j += 1
    ans = max(ans, j - i)
    cnt[ord(s[i]) - 97] -= 1
    i += 1
print(ans)
```

基本信息

#: 51984224
题目: E29950
提交人: 25n2500011619
内存: 3740kB
时间: 74ms
语言: Python3
提交时间: 2026-03-15 15:37:43

M30218:狭路相逢

stack, <http://cs101.openjudge.cn/practice/M30218/>

思路:

用栈 stack 存储还活着的人员，如果新进来的是怪物（负值），就让它一直消耗栈顶的勇士直到勇士死光或怪物死光（变成非负），然后把剩下的值加进栈中。

代码:

```
n = int(input())
a = list(map(int, input().split()))
stack = []
for x in a:
    while x < 0 and stack and stack[-1] > 0:
        y = stack.pop()
        x = x + y
    if x:
        stack.append(x)
print(len(stack))
print(' '.join(map(str, stack)))
```

代码运行截图 (至少包含有"Accepted")

#51987405提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
n = int(input())
a = list(map(int, input().split()))
stack = []
for x in a:
    while x < 0 and stack and stack[-1] > 0:
        y = stack.pop()
        x = x + y
    if x:
        stack.append(x)
print(len(stack))
print(' '.join(map(str, stack)))
```

基本信息

#: 51987405
题目: M30218
提交人: 25n2500011619
内存: 3680kB
时间: 26ms
语言: Python3
提交时间: 2026-03-15 19:58:52

M02299: Ultra-QuickSort

merge sort, <http://cs101.openjudge.cn/practice/02299/>

思路:

归并排序求逆序对

代码:

```
import sys

def merge_sort(arr, tmp, l, r) -> int:
    if l == r:
        return 0
    mid = (l + r) >> 1
    ans = merge_sort(arr, tmp, l, mid) + merge_sort(arr, tmp, mid + 1, r)

    for k in range(l, r + 1):
        tmp[k] = arr[k]
    i, j, k = l, mid + 1, l

    while i <= mid and j <= r:
        if tmp[i] <= tmp[j]:
            arr[k] = tmp[i]
            i += 1
        else:
            ans += mid - i + 1
            arr[k] = tmp[j]
            j += 1
        k += 1

    while i <= mid:
```

```

        arr[k] = tmp[i]
        k += 1
        i += 1
# 这部分本身已经有序
# while j <= r:
#     arr[k] = tmp[j]    #     k += 1    #     j += 1    return ans

def main():
    data = sys.stdin.read().split()
    it = iter(data)
    while True:
        n = int(next(it))
        if not n:
            break
        arr = [int(next(it)) for _ in range(n)]
        tmp = [0] * n
        print(merge_sort(arr, tmp, 0, n - 1))

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#51987808提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

import sys

def merge_sort(arr, tmp, l, r) -> int:
    if l == r:
        return 0
    mid = (l + r) >> 1
    ans = merge_sort(arr, tmp, l, mid) + merge_sort(arr, tmp, mid + 1, r)
    for k in range(l, r + 1):
        tmp[k] = arr[k]
    i, j, k = l, mid + 1, l
    while i <= mid and j <= r:
        if tmp[i] <= tmp[j]:
            arr[k] = tmp[i]
            k += 1
            i += 1
        else:
            ans += mid - i + 1
            arr[k] = tmp[j]
            k += 1
            j += 1
    for k in range(l, r + 1):
        arr[k] = tmp[k]
    return ans

```

基本信息

#: 51987808
 题目: 02299
 提交人: 25n2500011619
 内存: 69632kB
 时间: 3116ms
 语言: Python3
 提交时间: 2026-03-15 20:30:20

M29954:逃离紫罗兰监狱

bfs, <http://cs101.openjudge.cn/practice/29954>

思路:

用 dp 存储状态 x, y, k_left (剩下的魔法次数) 的最短步数, 然后用 bfs 结合双端队列处理。可以使用剪枝:

- 如果 $dist = ans - 1$, 那么任何后续路径长度都 $\geq ans$
- 不可能比当前最优答案 ans 更好
- 所以可以直接跳过

时间复杂度: $O(\text{BFS处理的状态数} \times 4) = O(r \times c \times \min(k, D))$, 其中 D 是到达 E 的最短距离。

代码:

```
from collections import deque

def main():
    r, c, k = map(int, input().split())
    grid = []
    sx, sy = 0, 0
    dx = [1, 0, -1, 0]
    dy = [0, 1, 0, -1]
    for i in range(r):
        s = input()
        for j, ch in enumerate(s):
            if ch == 'S':
                sx, sy = i, j
        grid.append(s)
    inf = float('inf')
    dp = [[[inf] * (k + 1) for j in range(c)] for i in range(r)]
    dp[sx][sy][k] = 0
    q = deque([(sx, sy, k, 0)])
    ans = inf
    while q:
        ux, uy, walls_left, dist = q.popleft()
        if dist > dp[ux][uy][walls_left] or dist >= ans - 1: continue
        dist += 1
        for t in range(4):
            nx = ux + dx[t]
            ny = uy + dy[t]
            if 0 <= nx < r and 0 <= ny < c:
                nk = walls_left
                if grid[nx][ny] == '#':
                    nk -= 1
                if nk < 0: continue
                if dp[nx][ny][nk] > dist:
                    dp[nx][ny][nk] = dist
                    if grid[nx][ny] == 'E':
                        ans = min(ans, dist)
```

```

        continue
        q.append((nx, ny, nk, dist))
    print(-1 if ans == inf else ans)

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#51988142提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```

from collections import deque

def main():
    r, c, k = map(int, input().split())
    grid = []
    sx, sy = 0, 0
    dx = [1, 0, -1, 0]
    dy = [0, 1, 0, -1]
    for i in range(r):
        s = input()
        for j, ch in enumerate(s):
            if ch == 'S':
                sx, sy = i, j
        grid.append(s)
    inf = float('inf')
    dp = [[[inf] * (k + 1) for j in range(c)] for i in range(r)]
    dp[sx][sy][k] = 0
    q = deque([(sx, sy, k, 0)])
    ans = inf
    while q:
        nx, ny, nk, nd = q.popleft()

```

基本信息

#: 51988142
 题目: 29954
 提交人: 25n2500011619
 内存: 41416kB
 时间: 134ms
 语言: PyPy3
 提交时间: 2026-03-15 20:59:47

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

第一次月考没参加，自己课后完成。前四题相对顺利的完成，Ultra-QuickSort尝试用归并排序但是遇到空间不够的问题，在AI的提示下改成重复使用数组空间而不是反复开新数组，解决了空间问题。BFS的题是一个经典问题，需要把魔法数作为一个维度记入搜索的状态，结合记忆化搜索（DP）解决。

这周学习了KMP算法，同时试着根据“利用对称位置信息”实现了Manacher算法，在写的过程中对这两个算法的深层逻辑感受更深。都是利用结构本身的‘对称’信息，KMP是用lsp数组表征每个前缀/后缀子串的匹配信息，事实上是预先先把pattern串的每个前缀串自己与自己做了一次匹配，那么与text的匹配一旦失配，那么text已匹配的部分就是信息已知的pattern子串，可以根据lsp信息预知下一个可能的匹配位置。回文串的对称我感觉更直白一点，就是关于某个中心center的左右对称，直接把左边已经判断过的部分复制到右边再继续扩展即可。